



MongoDB : un écosystème en évolution

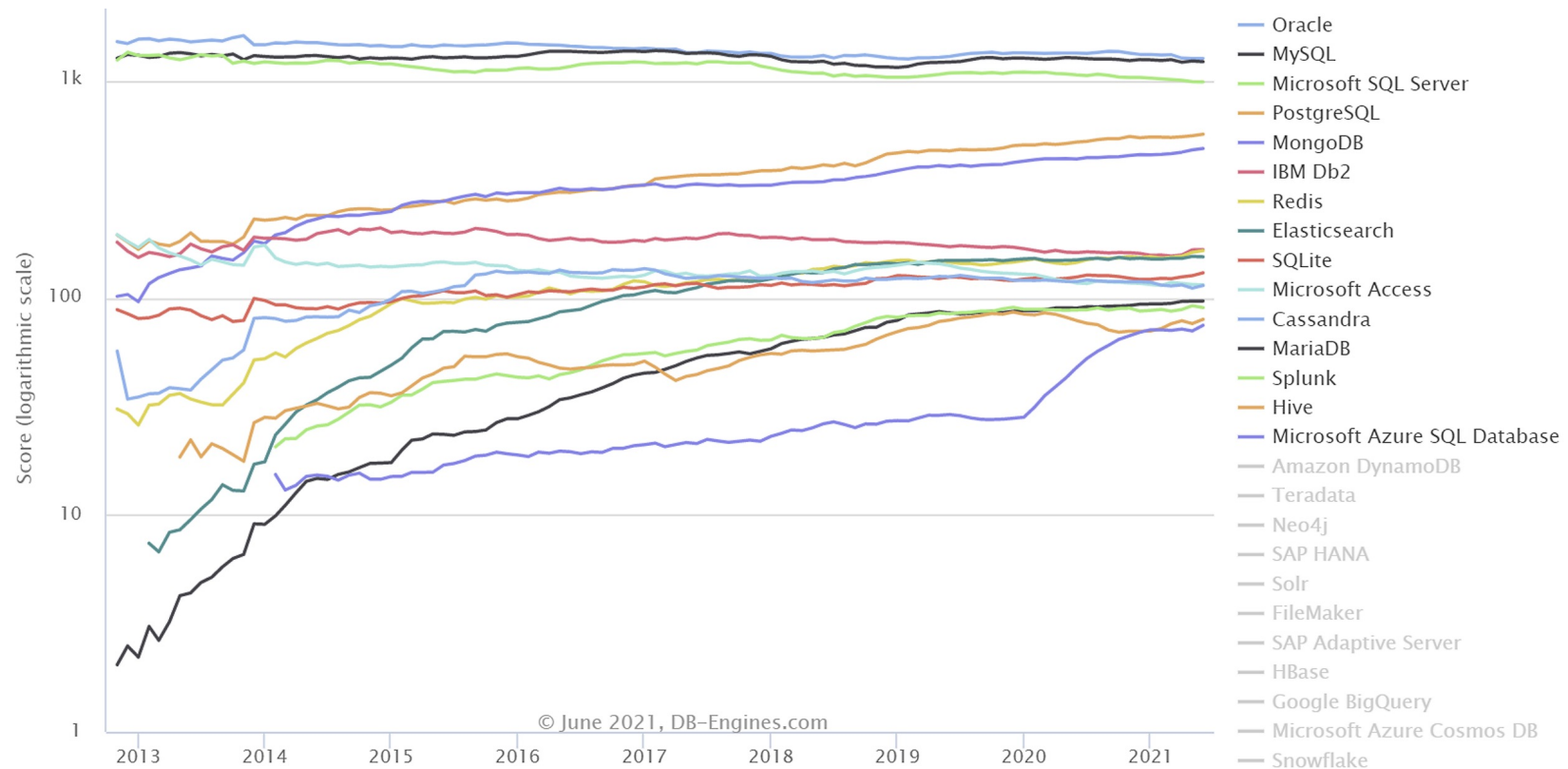
Dans un marché NoSQL très concurrentiel, MongoDB adopte une stratégie d'innovation continue :

- enrichissement régulier des fonctionnalités : réplication, sharding, transactions, sécurité, recherche full-text et vectorielle ;
- outils complémentaires comme MongoDB Compass ;
- développement de MongoDB Atlas, son offre Cloud managée ;
- services et support pour les entreprises.

MongoDB évolue ainsi d'une base principalement documentaire vers une plateforme proposant des services de données de plus en plus diversifiés.

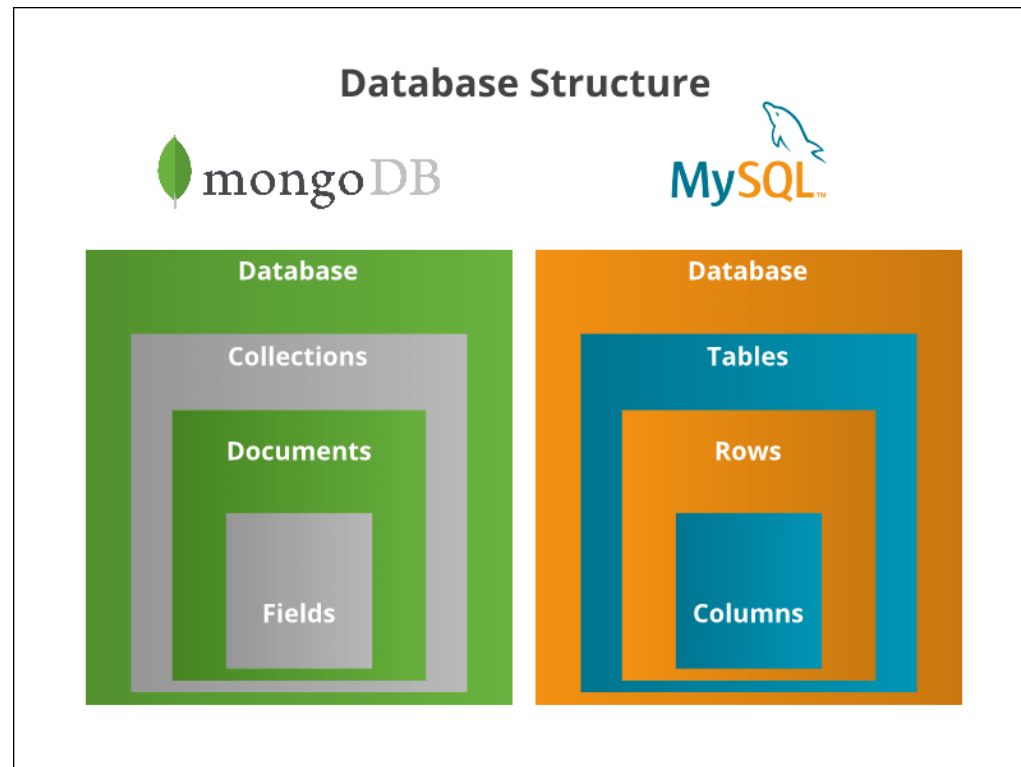


Classement tendance (trend) des moteurs



La sémantique MongoDB

MongoDB



Structure d'un document Json

Exemple de document JSON

```
{
  "id" : 123,
  "nom" : "Dupont",
  "actif" : true,
  "taille" : 1.78,
  "date_inscription" : "2025-09-20",
  "tags" : ["recherche", "données", "JSON"],
  "adresse" : {
    "rue" : "10 rue des Sciences",
    "ville" : "Paris",
    "code_postal" : "75005"
  },
  "projets" : [
    {
      "nom" : "rBDD",
      "role" : "membre"
    },
    {
      "nom" : "OpenData",
      "role" : "contributeur"
    }
  ]
}
```

Types scalaires (valeurs simples)

Les types scalaires représentent une seule valeur.

Chaîne de caractères (string)	"Dupont"	Entre guillemets
Nombre entier (number)	123	Sans guillemets
Nombre décimal (number)	1.78	Avec un séparateur .
Booléen (boolean)	true	true ou false
Valeur nulle (null)	null	Représente l'absence de valeur

Tableau (array)

Un tableau est une liste ordonnée de valeurs, qui peuvent être des scalaires, des documents ou un mélange des deux.

`["recherche", "données", "JSON"]`

Tableau de chaînes de caractères



Document (objet)

Un document (ou objet) est un ensemble de paires clé / valeur.

```
{
  "rue" : "10 rue des Sciences",
  "ville" : "Paris",
  "code_postal" : "75005"
}
```

Les valeurs d'un document peuvent elles-mêmes être des scalaires, des tableaux ou d'autres documents.



Tableau de documents

Un tableau peut contenir plusieurs documents.

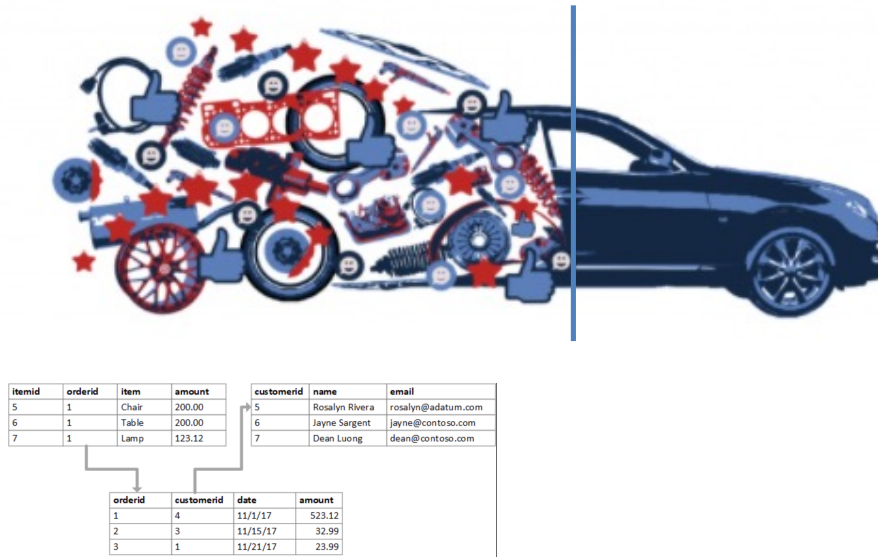
```
[
  { "nom" : "rBDD", "role" : "membre" },
  { "nom" : "OpenData", "role" : "contributeur" }
]
```



Modèle relationnel vs modèle Document

Introduction au modèle orienté document

Modèle de données Relationnel



Représentation logique sous forme de tables à 2 dimensions

Modèle de donnée orienté Document



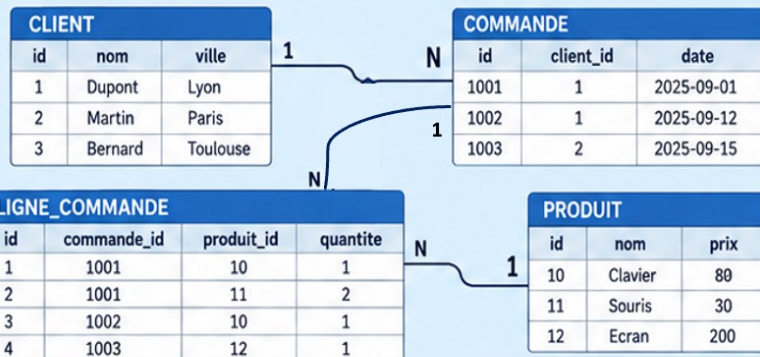
Représentation logique sous forme de documents

Modèle relationnel vs modèle Document

Modèle relationnel

Des données réparties dans plusieurs tables

Les informations d'un même objet métier sont stockées dans plusieurs tables et liées par des clés. Pour reconstruire l'objet complet, il faut effectuer des jointures.



Pour obtenir une commande complète :

- Plusieurs jointures sont nécessaires
- Les données sont réparties dans différentes tables



Avantages du modèle relationnel



Intégrité des données

Contraintes (clés, unicité, référentielle).



Transactions ACID

Garantit la cohérence même en cas d'erreur ou de concurrence.



Requêtes complexes

Jointures, agrégations, analyses fines.



Maturité et écosystème riche

Outils, normes, retours d'expérience.

Inconvénients



Schéma rigide

Évolution plus complexe.



Multiples jointures

Peut impacter les performances et la scalabilité.



Distribution plus complexe

Nécessite des mécanismes sophistiqués pour répartir les données.



Moins adapté aux données dynamiques ou très variées.

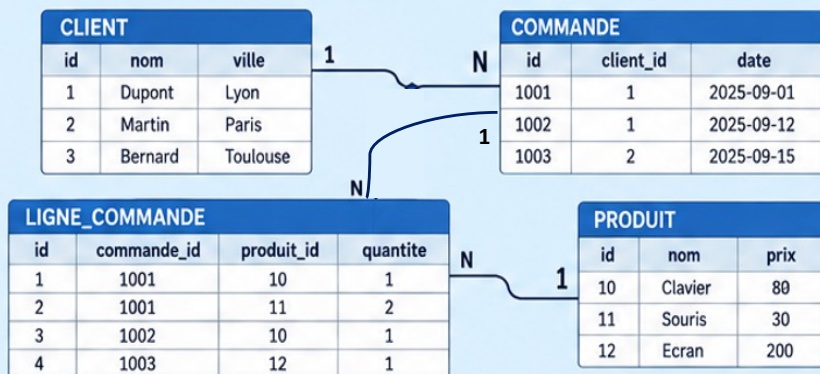
Modèle relationnel vs modèle Document

2 façons d'organiser les données, 2 approches pour des besoins différents

Modèle relationnel

Des données réparties dans plusieurs tables

Les informations d'un même objet métier sont stockées dans plusieurs tables et liées par des clés. Pour reconstruire l'objet complet, il faut effectuer des jointures.



Même information
mais une
organisation
différente



Modèle document

Un document auto-suffisant

Les données fréquemment utilisées ensemble sont regroupées dans un même document (dénormalisation). Une seule lecture permet de récupérer l'objet complet, sans jointure.

Exemple de document JSON (une commande)

```
{
  "id": 1001,
  "client": {
    "id": 1,
    "nom": "Dupont",
    "ville": "Lyon"
  },
  "date": "2025-09-01",
  "produits": [
    { "id": 10, "nom": "Clavier", "prix": 80, "quantite": 1 },
    { "id": 11, "nom": "Souris", "prix": 30, "quantite": 2 }
  ],
  "total": 140
}
```

✓ Un document autonome

- ✓ Toutes les informations utiles sont contenues dans le document
- ✓ Une seule lecture suffit
- ✓ Peu ou pas de jointures dans les cas d'usage courants

✗ Pour obtenir une commande complète :

- Plusieurs jointures sont nécessaires
- Les données sont réparties dans différentes tables



Modèle relationnel vs modèle Document

Modèle document

Un document auto-suffisant

Les **données fréquemment** utilisées ensemble sont regroupées dans un même document (dénormalisation).
Une seule lecture permet de récupérer l'objet complet, sans jointure.

Exemple de document JSON (une commande)

```
{
  "id": 1001,
  "client": {
    "id": 1,
    "nom": "Dupont",
    "ville": "Lyon"
  },
  "date": "2025-09-01",
  "produits": [
    { "id": 10, "nom": "Clavier", "prix": 80, "quantite": 1 },
    { "id": 11, "nom": "Souris", "prix": 30, "quantite": 2 }
  ],
  "total": 140
}
```

✓ Un document autonome

- ✓ Toutes les informations utiles sont contenues dans le document
- ✓ Une seule lecture suffit
- ✓ Peu ou pas de jointures dans les cas d'usage courants

Avantages du modèle document



Passage à l'échelle facilité
Documents autonomes, faciles à distribuer (sharding).



Modèle flexible
Pas de schéma fixe, s'adapte aux évolutions.



Moins (ou pas) de jointures
Une lecture suffit pour accéder à l'objet complet.

Inconvénients



Pas (ou peu) transactionnel
Pas de transactions ACID multi-documents (nativement).
Les transactions existent mais sont limitées (en périmètre et en performances).



Risque de duplication
Les données peuvent être dupliquées entre documents (cohérence à gérer).



Moins adapté à certaines contraintes fortes d'intégrité et de cohérence globale.

Modèle relationnel vs modèle Document

En regroupant les données liées dans un même document :

- on limite les jointures
- on favorise la localité des données
- on facilite la distribution à grande échelle.

Distribution des données

Dans une base relationnelle

Les données liées sont réparties dans différentes tables, ce qui complique la distribution et peut nécessiter des échanges entre machines pour réaliser les jointures.



Dans une base orientée documents

Chaque document est une unité de données autonome qui peut être stockée sur n'importe quelle machine. Les requêtes sont majoritairement locales.





CRUD : Create

Comment créer un document dans une base mongo ?

```
db.products.insertOne( { item: "card", qty: 15 } )
```

db : objet de connexion à la bdd

Product : nom de la collection

insertOne : action d'insertion d'un seul document



MongoDB collection **products**

```
{ "_id" : ObjectId("5063114bd386d8fadbd6b004"), "item" : "card", "qty" : 15 }
```

Chaque document possède un identifiant unique (_id).

S'il n'est pas précisé lors de l'insertion du document, MongoDB le génère automatiquement.

CRUD : Create

```
db.products.insertMany(  
  [  
    { item: "pen", qty: 20 },  
    { _id: 11, item: "pencil", qty: 50, type: "no.2" },  
    { item: "eraser", qty: 25 }  
  ]  
)
```



```
{ "_id" : ObjectId("51e0373c6f35bd826f47e9a0"), "item" : "pen", "qty" : 20 }  
{ "_id" : 11, "item" : "pencil", "qty" : 50, "type" : "no.2" }  
{ "_id" : ObjectId("51e0373c6f35bd826f47e9a1"), "item" : "eraser", "qty" : 25 }
```

Lors d'une insertion multiple, si une erreur survient lors de l'insertion d'un document, MongoDB conserve les documents déjà insérés : il n'y a pas de rollback automatique de l'ensemble de l'opération.



CRUD : Read

```
db.collection.find( {condition},{projection}) return cursor
```

La condition est exprimée sous forme de document : { "champ" : valeur , ... }

Rechercher tous les restaurants dans le quartier (borough) de Brooklyn:

```
db.restaurants.find( { "borough" : "Brooklyn" } )
```

Rechercher les restaurants dans le quartier (borough) de Brooklyn **ET** dont la cuisine est italienne.

```
db.restaurants.find( { "borough" : "Brooklyn", "cuisine" : "Italian" } )
```

CRUD : Read

Le 2^e paramètre de `find()` permet de sélectionner les champs à retourner.

```
db.collection.find(  
  { filtre },  
  { champ1: 1, champ2: 1, _id: 0 }  
)
```

Champ1 : 1 / true → le résultat **affiche** le champ champ1
_id : 0 / false → le résultat n'**affiche pas** le champ _id

```
db.restaurants.find(  
  { "borough" : "Brooklyn", "cuisine" : "Italian" },  
  {borough:1}  
)
```

Affiche uniquement le champs borough de chaque document : par défaut _id est affiché.

```
db.restaurants.find(  
  { "borough" : "Brooklyn", "cuisine" : "Italian" },  
  {borough:0}  
)
```

Affiche tout les champs du document satisfaisant la condition à l'exception du champs borough (borough:0)

CRUD : Read

```
{
  "_id" : 1,
  "name" : "sue",
  "age" : 19,
  "type" : 1,
  "status" : "P",
  "favorites" : { "artist" : "Picasso", "food" : "pizza" },
  "finished" : [ 17, 3 ]
  "badges" : [ "blue", "black" ],
  "points" : [ { "points" : 85, "bonus" : 20 }, { "points" : 85, "bonus" : 10 } ]
}
```

Quelle requête retourne le document ci-dessus :

- `db.users.find( { badges: "black" }, { name: 1 } )`
- `db.users.find( { "badges.0": "black", { points: 0 } } )`
- `db.users.find( { "favorites.food": "Picasso" }, { "points.0.bonus": 0 } )`
- `db.users.find( { statu: "A", age: { $lt: 30 } } ).skip(0).limit(5)`

CRUD : Read

```
db.addressBook.insertMany(  
  [  
    { "_id" : 1, address : "2030 Martian Way", zipCode : "90698345", "contact" : true },  
    { "_id" : 2, address : "156 Lunar Place", zipCode : 43339374, "contact" : false },  
    { "_id" : 3, address : "2324 Pluto Place", zipCode : NumberLong(3921412), "contact" : false },  
    { "_id" : 4, address : "55 Saturn Ring", zipCode : NumberInt(88602117) },  
    { "_id" : 5, address : "104 Venus Drive", zipCode : ["834847278", "1893289032"], "contact" : true },  
    { "_id" : 6, address : "222 Sun Boulevard", zipCode : null, "contact" : true }  
  ]  
)
```

db.addressBook.find({ "zipCode" : { \$type : "string" } }) => {1,5}

db.addressBook.find({ "zipCode" : { \$not : { \$type : "null" } } }) => {1,2,3,4,5}

db.addressBook.find({ "contact" : { \$exists : true } }) => {1,2,3,5,6}

db.addressBook.find({ "contact" : { \$exists : true, \$eq : true } }) => {1,5,6}



CRUD : Read

- Opérateurs de comparaisons : \$eq, \$gt, \$gte, \$in \$nin
db.inventory.find({ qty: { \$nin: [5, 15] } })
<https://docs.mongodb.com/manual/reference/operator/query-comparison/>
- Opérateurs logiques : \$and, \$or, \$not, \$nor
db.inventory.find({ price: { \$not: { \$gt: 1.99 } } })
<https://docs.mongodb.com/manual/reference/operator/query-logical/>
- Opérateur sur les éléments : \$exists, \$type
db.inventory.find({ price: { \$type : "double" } })
<https://docs.mongodb.com/manual/reference/operator/query-element/>
- Opérateur de projection
Db. inventory.find({ points: { \$elemMatch: {point: { \$gt: 70 },bonus: { \$gt:90 } } } },{ "grades.\$": 1 })
<https://www.mongodb.com/docs/manual/reference/operator/projection/>



CRUD : Read

- Une requête peut retourner **des centaines, des milliers, voire des millions** de documents.
- MongoDB ne retourne pas nécessairement tous les résultats en une seule fois. `find()` retourne un **curseur**, qui permet de parcourir les résultats **progressivement, par lots (batches)**.
- Cela permet notamment de :
 - limiter la **mémoire utilisée** ;
 - limiter les **transferts réseau** ;
 - traiter progressivement de grands volumes de résultats.

Exemple :

- `limit()`: contraint la taille de l'ensemble résultat
- `count()`: modifie le curseur pour retourner le nombre de documents plutôt que les documents eux-mêmes.
- `skip()`: retourne un curseur qui pointe sur les documents résultat après avoir passé le nombre de documents indiqué.
- `pretty()`: configure le curseur pour afficher les documents de manière plus facile à lire (pour un humain)

```
db.trips.find({"usertype" : "Customer"}).count()  
db.zips.find({"state" : "CA"}).skip(500).limit(10)
```

La liste complète :

<https://docs.mongodb.com/manual/reference/method/js-cursor/>

CRUD : Update

■ MongoDB propose plusieurs méthodes de mise à jour :

`Db.mycollection.updateOne({$query},{ $set})` / modifie **un seul document**, même si plusieurs correspondent au filtre.

`Db.mycollection.updateMany({$query},{ $set})` → modifie **tous les documents** correspondant au filtre.

`db.collection.replaceOne({$query},{replacement})` → **remplace entièrement** un document.

`db.collection.replaceMany({$query},{replacement})` → **remplace entièrement** plusieurs documents.

```
db.users.updateOne(  
  { ville: "Lyon" },  
  { $set: { actif: true } }  
)
```

```
db.restaurant.replaceMany(  
  { "name" : "Central Perk Cafe" },  
  { "name" : "Central Pork Cafe", "Borough" : "Manhattan" }  
)
```



CRUD : Update

- Une opération portant sur un seul document est atomique.
- Une opération `updateMany()` est atomique **document par document**, mais l'ensemble de l'opération n'est pas une transaction atomique.
- Pour garantir une modification **tout ou rien** sur plusieurs documents, il faut utiliser une **transaction multi-documents**.

Delete

```
db.collection.deleteOne(filtre)
```

-> Supprime **un seul document** vérifiant le filtre. Si plusieurs documents correspondent, **un seul est supprimé**.

```
db.collection.deleteMany(filtre)
```

-> Supprime **tous les documents** correspondant au filtre.

```
db.products.deleteOne({  
  item: "pencil",  
  type: "blue"  
})
```



```
{ "acknowledged": true, "deletedCount": 1 }
```

```
db.products.deleteMany({  
  type: "blue"  
})
```



```
{ "acknowledged": true, "deletedCount": 5 }
```

TP 01 MONGO CRUD

